Session 1: Introduction; the Lambda Calculus

What Structure Does Meaning Composition Require?

James Pustejovsky

Brandeis University
Department of Computer Science

CS 135: Computational Semantics

August 28, 2026

Today

1. A question to start with
2. What this course is, how it is assessed, what you need
3. **The arc** – five bodies of material, and why each one forces the next
4. The lambda calculus – as far as the clock allows

At the end: a paper handout, a linear algebra self-assessment. Not graded. Please take one on your way out.

Two Sentences

The dog bit the man.

The man bit the dog.

Same words. Same word frequencies. Same bag of tokens.

Two Sentences

The dog bit the man.

The man bit the dog.

Same words. Same word frequencies. Same bag of tokens.

Radically different meanings. One of them is news.

Two Sentences

The dog bit the man.

The man bit the dog.

Same words. Same word frequencies. Same bag of tokens.

Radically different meanings. One of them is news.

The question for this course: what does a representation of meaning have to *be* in order to tell these apart – and to keep telling them apart when the sentences get complicated?

Why This Is Harder Than It Looks

Suppose meanings are vectors – as they are in essentially every modern NLP system. How do we combine the vectors for *dog*, *bit*, and *man*?

Operation	**Problem**
$\mathbf{u} + \mathbf{v}$	commutative – both sentences get the same vector
$\mathbf{u} \odot \mathbf{v}$	also commutative – same problem
$[\mathbf{u}; \mathbf{v}]$	order-sensitive, but dimension grows with sentence length
$\mathbf{u} \otimes \mathbf{v}$	order-sensitive, but dimension grows *exponentially*

Why This Is Harder Than It Looks

Suppose meanings are vectors – as they are in essentially every modern NLP system. How do we combine the vectors for *dog*, *bit*, and *man*?

Operation	**Problem**
$\mathbf{u} + \mathbf{v}$	commutative – both sentences get the same vector
$\mathbf{u} \odot \mathbf{v}$	also commutative – same problem
$[\mathbf{u}; \mathbf{v}]$	order-sensitive, but dimension grows with sentence length
$\mathbf{u} \otimes \mathbf{v}$	order-sensitive, but dimension grows *exponentially*

Meanwhile the logical tradition solves ordering immediately and completely – but has no notion of *similarity* at all. In it, *dog* and *wolf* are simply different symbols.

Why This Is Harder Than It Looks

Suppose meanings are vectors – as they are in essentially every modern NLP system. How do we combine the vectors for *dog*, *bit*, and *man*?

Operation	**Problem**
$\mathbf{u} + \mathbf{v}$	commutative – both sentences get the same vector
$\mathbf{u} \odot \mathbf{v}$	also commutative – same problem
$[\mathbf{u}; \mathbf{v}]$	order-sensitive, but dimension grows with sentence length
$\mathbf{u} \otimes \mathbf{v}$	order-sensitive, but dimension grows *exponentially*

Meanwhile the logical tradition solves ordering immediately and completely – but has no notion of *similarity* at all. In it, *dog* and *wolf* are simply different symbols.

Each tradition solves exactly what the other cannot. That is the situation this course is about.

What This Course Is

One question, pursued across five bodies of material:

	Theme	**The idea**
1	Meaning as logic	composition is function application
2	Meaning as distribution	meaning is position in a space
3	Linear algebra, revisited	the geometry we already half-know
4	Geometric algebra	one product that keeps what the others lose
5	Functional geometric algebra	semantics on that substrate

This is not a survey of five topics. It is one argument in five movements, and each movement exists because the previous one ran out of road.

Learning Goals

By the end of the course you will be able to:

- represent and compute meaning in both logical and distributional frameworks, and articulate the strengths and limits of each;
- carry out type-driven semantic derivations, including quantification and scope, and evaluate them against models;
- implement a compositional fragment of English and test it computationally;
- work with the core constructions of geometric algebra and apply them to semantic representation;
- carry out and evaluate mappings between model-theoretic artifacts and vectorial encodings;
- read and evaluate contemporary research in computational semantics – including work whose central questions are still open;
- integrate these skills in a substantial final project.

Prerequisites – and a Reassurance

Formally: introductory programming (any language), plus LING 130a or COSI 121b, or permission of the instructor.

In practice, two things matter:

- **No prior Haskell or functional programming is assumed.** Problem Set 1 gives you a working file with holes in it; you write the *semantics*, not the infrastructure. Nearly every answer is one line. Labs 1–3 cover the language.
- **Some linear algebra helps in the second half** – not the computational kind, but you should be comfortable with vectors, inner products, and what a determinant means geometrically. Hence today's handout.

Students arrive here from computer science, from linguistics, and from elsewhere. One of the five themes will probably be much harder for you than the others. That is expected, and it is not a signal that you are in the wrong room. It is a signal to come to office hours early.

Resources

No textbook to buy. Everything is on Moodle or freely available online.

Key	Text	Used in
EU	van Eijck & Unger, *Comp. Semantics with Functional Programming*	Theme 1
JM	Jurafsky & Martin, *Speech and Language Processing* (3rd ed.)	Themes 1–2
LS	Lenci & Sahlgren, *Distributional Semantics*	Theme 2
W	Widdows, *Geometry and Meaning*	Themes 2–3
D	Dorst, Fontijne & Mann, *Geometric Algebra for Computer Science*	Theme 4
M	Macdonald, *Linear and Geometric Algebra* (optional, gentler)	Themes 3–4
P	Papers packet – Smolensky, Plate, Coecke et al., FGA papers	Themes 4–5

Course site: `www.cs-135.org` – schedule, slides, assignments, announcements.
Moodle: readings and submissions.

Assessment

Component	Weight
Problem Set 1	15%
Problem Set 2	15%
Problem Set 3	15%
Quiz 1 (take-home)	10%
Quiz 2 (take-home)	10%
Final project	30%
Participation and lab	5%

The three problem sets are one argument:

1. Build a fragment of English in the logical tradition.
2. Build distributional representations – and *demonstrate for yourself* where they fail to compose.
3. Rebuild the fragment from PS 1 on a geometric substrate; evaluate the comparison.

PS 3's writeup asks what the geometric version *cannot* do. A well-argued negative result earns full credit.

The Semester at a Glance

Sessions	Part	Theme	Assessment
1–6	Meaning as Logic	1	PS 1
7–10	Meaning as Distribution	2	Quiz 1, PS 2
11	The algebraic thread in NLP	2–3	
12–16	Geometric Algebra (+ flex)	3–4	Quiz 2
17–24	Functional Geometric Algebra	5	PS 3
25–27	Projects	—	Final project

Session 16 is a deliberate flex session. Sessions 12–15 build a lot of mathematics quickly. If it is not landing, tell me *before* we get there and we slow down.

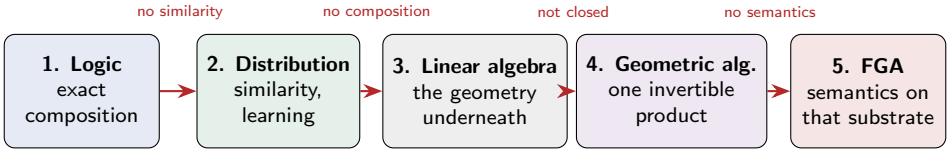
Session 24 is the research frontier. Some of what we discuss there has no published answer. Final projects may take those questions on, and are graded on the quality of the investigation, not on whether the approach works.

Administration over.

The Arc

Five themes. Each one exists because the last ran out of road.

The Arc in One Picture



Red labels are **what is missing** – the gap that forces the next move.

We now walk this arc once, quickly, so you know where we are going. Then we go back to the start and do it properly.

Frege's Principle (compositionality)

The meaning of a complex expression is determined by the meanings of its parts and the way they are combined.

Take this seriously and two questions follow:

1. What *are* the meanings of the parts?
2. What is the combination operation?

Frege's Principle (compositionality)

The meaning of a complex expression is determined by the meanings of its parts and the way they are combined.

Take this seriously and two questions follow:

1. What *are* the meanings of the parts?
2. What is the combination operation?

Montague's answer (1970), which has organized formal semantics ever since:

The parts are functions. The combination operation is function application.

Theme 1: How It Works

Every expression gets a **semantic type** recording what it combines with.

Expression	Type	Denotation
Alice	e	an individual
sleeps	$\langle e, t\rangle$	individuals to truth values
likes	$\langle e, \langle e, t\rangle\rangle$	individuals to predicates
Alice sleeps	t	a truth value

Composition is application, and the types say which way round:

$$[\![\text{likes Bob}]\!] = (\lambda x.\lambda y.\,\text{like}(y, x))(\text{b}) \;\rightsquigarrow\; \lambda y.\,\text{like}(y, \text{b})$$
$$[\![\text{Alice likes Bob}]\!] = (\lambda y.\,\text{like}(y, \text{b}))(\text{a}) \;\rightsquigarrow\; \text{like}(\text{a}, \text{b})$$

And *Bob likes Alice* gives $\text{like}(\text{b}, \text{a})$ – a different formula. The opening problem, solved exactly, in two steps.

Theme 1: What It Buys

- **Exact structure.** Argument positions are never confused. β-reduction loses nothing.
- **Unbounded recursion.** Embedding is free: *Alice thinks Bob said Carol left*.
- **Truth conditions.** A sentence's meaning connects to a model – so we can ask whether it is *true*, and what follows from it.
- **Entailment and inference.** *Every student sleeps* $\Rightarrow$ *Alice sleeps*, if Alice is a student. This falls out of the semantics rather than being stipulated.
- **A theory of what cannot combine.** Type mismatch is not falsehood; it is undefinedness.

Sixty years of results, and everything we build later is measured against this.

Theme 1: Where It Runs Out

What the logical tradition cannot say:

- *dog* and *wolf* are **similar**. Here they are just distinct constants; there is no notion of degree.
- How to get $[\![\text{dog}]\!]$ from **data**. The lexicon is written by hand.
- What a word means **in context** – *bank* by the river vs. *bank* with the money.
- Anything **graded**: typicality, vagueness, gradient acceptability.

The gap

Meaning here is *structural* but not *substantive*. We know how dog composes; we know nothing about what dog is like.

This forces Theme 2. If you want similarity and learnability, meanings must live in a space with a metric – not be symbols in a lexicon.

The distributional hypothesis

Words that occur in similar contexts tend to have similar meanings. *(Harris 1954; Firth 1957)*

A radical proposal: stop writing lexical entries by hand. **Read meaning off the corpus.**

	barked	*purred*	*drove*	*ate*
dog	42	0	1	38
cat	2	51	0	44
driver	0	0	63	12

Each row is a vector. *dog* and *cat* are now *close* – and nobody had to say so.

Theme 2: The Machinery

1. **Raw counts** are dominated by frequency. Reweight – pointwise mutual information, then PPMI:
$$\mathrm{PMI}(w, c) = \log \frac{P(w, c)}{P(w)P(c)}$$
2. **The matrix is huge and sparse.** Reduce it – singular value decomposition, or learn a low-dimensional embedding directly (word2vec, GloVe).
3. **Similarity is geometry.** The cosine of the angle between two vectors:
$$\cos(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \, \|\mathbf{v}\|}$$
4. **Context can be built in.** A word's vector need not be fixed – transformers compute it from the sentence it appears in.

Notice that step 3 is already linear algebra doing semantic work. Hold that thought until Theme 3.

Theme 2: What It Buys

- **Similarity, for free.** Nearest neighbors of *dog*: *cat*, *puppy*, *pet*, *hound*.
- **Learnability.** No hand-written lexicon. Point it at a corpus and it improves.
- **Graded structure.** Typicality, vagueness, and degree come out naturally, because the space has a metric.
- **Context sensitivity.** Contextual embeddings give *bank* different vectors in different sentences.
- **It works.** Every deployed NLP system in the world is built on this.

Everything Theme 1 could not do, Theme 2 does easily.

Theme 2: Where It Runs Out

Return to the opening problem. We have $\mathbf{dog}$, $\mathbf{bit}$, $\mathbf{man}$. Combine them.

Operation	Keeps	Loses
$\mathbf{u} + \mathbf{v}$	topic	order, roles
$\mathbf{u} \odot \mathbf{v}$	shared features	order, roles; sparse overlap
$[\mathbf{u}; \mathbf{v}]$	order	fixed dimension; no recursion
$\mathbf{u} \otimes \mathbf{v}$	order, roles	fixed dimension – d^2, then $d^3 \ldots$

The gap – the hinge of this course

Vector operations either *forget structure* or *explode in dimension*. Nothing on the standard toolkit binds two meanings, keeps their roles distinct, allows the binding to be undone, and returns something the same size as what went in.

This forces Theme 3. If the problem is which operations we have on vectors, the honest response is to ask what operations vectors actually admit.

Theme 3: Linear Algebra, Revisited

You have met most of this before. We revisit it because the standard presentation **hides the parts we need**.

Object	**Usually taught as**	**What it really is**
vector	a list of numbers	a directed magnitude, basis-independent
inner product	a formula, $\sum u_i v_i$	how much of $\mathbf{u}$ lies along $\mathbf{v}$
determinant	a computation	an oriented area or volume
matrix	a grid	a record of where the basis went

The right-hand column is what Themes 4 and 5 are built on. The left-hand column is why the connection is invisible to most people who have taken linear algebra.

This is also why today's handout weights determinants and orientation more heavily than a normal review would.

Theme 3: The Inner Product Loses Information

$$\mathbf{u} \cdot \mathbf{v} = \|\mathbf{u}\| \, \|\mathbf{v}\| \cos\theta$$

The inner product takes two vectors and returns **one number**. It measures alignment.

- It is **symmetric**: $\mathbf{u} \cdot \mathbf{v} = \mathbf{v} \cdot \mathbf{u}$. Order information is destroyed.
- It is **not invertible**: from $\mathbf{u} \cdot \mathbf{v} = 5$ you cannot recover $\mathbf{v}$.
- It **lowers dimension**: two vectors in, a scalar out.

This is the diagnosis from Theme 2 again – now stated about the *operation* rather than about the application.

So: is there an operation on vectors that is *not* symmetric, and that *can* be undone?

Theme 3: The Determinant Knows About Order

Here is the piece of linear algebra that matters most for this course – and it is usually taught as a computation.

$$\det \begin{pmatrix} a & b \\ c & d \end{pmatrix} = ad - bc$$

- Its **absolute value** is the area of the parallelogram spanned by the columns.
- Its **sign** records **orientation** – swap the columns and the sign flips.
- It is **zero** exactly when the columns are dependent.

An antisymmetric operation on two vectors returning an oriented magnitude. **That is order information, living inside linear algebra, in plain sight.**

Grassmann (1844) took this seriously and built an algebra from it: the **outer product** $\mathbf{a} \wedge \mathbf{b}$.

Theme 3: What We Have, and What Is Missing

	Symmetric?	Invertible?	Grade
Inner product $\mathbf{u} \cdot \mathbf{v}$	yes	no	lowers
Outer product $\mathbf{u} \wedge \mathbf{v}$	no	no	raises

The outer product keeps order. But it still cannot be undone – from $\mathbf{u} \wedge \mathbf{v}$ you cannot recover $\mathbf{v}$ knowing $\mathbf{u}$, because a plane does not determine its factors.

The gap

Two operations, each throwing away half of what we need, neither invertible. And the system is **not closed**: the outer product of two vectors is not a vector, so there is no single space in which we can keep working.

This forces Theme 4.

Clifford's move (1878): **do not choose between them**. Take the sum.

The geometric product

$$\mathbf{ab} = \underbrace{\mathbf{a}\cdot\mathbf{b}}_{\text{symmetric, scalar}} + \underbrace{\mathbf{a}\wedge\mathbf{b}}_{\text{antisymmetric, bivector}}$$

An object of mixed kind – a scalar plus an oriented area. That looks strange, and it is the entire point:

$$\mathbf{a}\,\mathbf{a}^{-1} = 1 \qquad \text{where} \qquad \mathbf{a}^{-1} = \frac{\mathbf{a}}{\|\mathbf{a}\|^2}$$

The geometric product is invertible. Neither of its parts is. Keeping both is exactly what makes division possible.

Theme 4: Grades

Close the vector space under this product and you get objects of every **grade**:

Grade	Name	Geometry	Example in $\mathbb{R}^3$
0	scalar	magnitude	3
1	vector	directed length	$2\mathbf{e}_1 - \mathbf{e}_3$
2	bivector	oriented area	$5\mathbf{e}_{12}$
3	trivector	oriented volume	$\mathbf{e}_{123}$

All of them live in **one algebra**, $\mathrm{Cl}(\mathbb{R}^n)$, and can be added and multiplied freely.

The two old products become grade operations:

- wedge $\mathbf{a} \wedge \mathbf{b}$ – **raises** grade: builds structure up
- contraction $\mathbf{a} \lrcorner B$ – **lowers** grade: takes structure apart

And these two are adjoint. Whatever the wedge builds, contraction is the canonical way to undo.

Theme 4: What It Buys

Return to the requirements the composition problem left us with:

Need	**Theme 2's attempts**	**Geometric algebra**
bind two meanings	$\otimes$ (explodes)	wedge $\wedge$
undo a binding	approximate, noisy	contraction $\lrcorner$ exact
preserve order	only via $\otimes$ or $[\,;]$	antisymmetry of $\wedge$
one carrier space	—	$\mathrm{Cl}(\mathbb{R}^n)$, all grades at once
transform continuously	—	rotors, $R \square \widetilde{R}$

Rotors deserve a note: $R = e^{-B\theta/2}$ applies a rotation by *multiplication*, preserves grade, composes, and is **differentiable in** θ.

A transformation you can differentiate is a transformation you can learn – which is how Theme 4 reconnects to Theme 2.

Theme 4: Where It Runs Out

Geometric algebra is a **mathematical** theory. It says nothing whatever about language.

Nothing in Clifford's work tells us:

- which multivector should represent *dog*;
- whether grade corresponds to anything linguistic;
- how quantification, negation, or coercion are supposed to work;
- what a *model* is in this setting, or what makes a sentence true.

The gap

We have an algebra with exactly the operations the composition problem demanded – and no interpretation. A substrate without a semantics.

This forces Theme 5 – and note that we are now back to the question Theme 1 answered, asked again over a different substrate.

Theme 5: Functional Geometric Algebra

The proposal: read the operations of the algebra as the operations of a semantics.

Semantics (Theme 1)		**Geometric algebra (Theme 4)**
arity of a predicate	$\longleftrightarrow$	**grade** of a multivector
argument binding	$\longleftrightarrow$	**wedge product** $\wedge$
function application	$\longleftrightarrow$	**left contraction** $\lrcorner$
type coercion	$\longleftrightarrow$	**rotor** action $R \,\square\, \widetilde{R}$
individuals	$\longleftrightarrow$	vectors (grade 1)
n-place predicates	$\longleftrightarrow$	grade-n structures

If this correspondence holds up, we get Theme 1's exact composition on Theme 2's graded, learnable substrate. That is the whole bet.

Theme 5: Grade as Arity

The central claim in one line: **a predicate's arity is the grade of the multivector representing it.**

	Type (Theme 1)	Grade (Theme 5)
Alice	e	1
sleeps	$\langle e, t \rangle$	2
likes	$\langle e, \langle e, t \rangle \rangle$	3

And application *lowers grade*, exactly as it lowers type:

$$\underbrace{3}_{\textit{likes}} \lrcorner \underbrace{1}_{\textit{Bob}} \longrightarrow \underbrace{2}_{\textit{likes Bob}} \lrcorner \underbrace{1}_{\textit{Alice}} \longrightarrow \underbrace{1}_{\text{truth}}$$

The same derivation you saw in Theme 1 – two applications, same order – but every object is now geometric, living in a metric space where similarity means something.

Theme 5: Coercion as Rotation

A place where the geometric picture offers something the logical one does not.

Type coercion in natural language:

Mary began the book. (began *reading* it? *writing* it?)

An object of one type is used where another is required, and the semantics repairs it.

In the logical tradition this needs a **type-shifting rule**: a discrete, stipulated operation.

In geometric algebra there is a natural candidate – a **rotor**:

$$M \longmapsto R\,M\,\widetilde{R}, \qquad R = e^{-B\theta/2}$$

- preserves grade – arity is unchanged ✓
- continuous in θ – coercion by degree, not by fiat
- composable and invertible

Whether this is the right analysis of coercion is exactly the sort of question this course leaves open.

Theme 5: What Is Actually Open

I want to be straightforward about the status of this material.

Settled	Geometric algebra itself – 150 years old, standard mathematics
Established	Distributional semantics; the composition problem
Proposed	The FGA correspondence: grade/arity, wedge/binding, contraction/application
Open	Does grade give full *typing*, or only arity?
	Is contraction-based derivation confluent?
	How is variable binding handled – reflexives, anaphora?
	What is a learning objective over graded representations?

Session 24 is about the last four rows, and final projects may attack any of them. You are not being asked to accept this framework – you are being asked to acquire the tools to **evaluate** it. Problem Set 3 explicitly asks you what it cannot do.

The Arc, Restated

	Theme	Gives us	Cannot give us
1	Logic	exact composition, truth, inference	similarity, learning
2	Distribution	similarity, learning, gradedness	composition
3	Linear algebra	the geometry underneath	a closed, invertible product
4	Geometric algebra	one invertible product; grades	any interpretation
5	FGA	a candidate semantics on that substrate	– still being worked out

Each row's last column is the next row's reason for existing.

We now go back to the top and start properly. Theme 1 begins with the machinery Montague borrowed from Church.

The Lambda Calculus

Church, 1936. The foundation of Theme 1 — and the thing Theme 5 tries to reconstruct.

Why This, Specifically?

We said: meanings are functions, composition is application. To make that precise and computable we need a notation in which

- functions can be **written down** without being named,
- functions can be **applied** by a purely mechanical rule,
- and functions can **take other functions** as arguments.

Church's lambda calculus does exactly this, with three constructors and one rule of computation.

It is also, not coincidentally, the foundation of functional programming – which is why Problem Set 1 is written in Haskell, and why the denotations you write there *are* the semantics rather than a program that computes it.

Lambda Terms: Syntax

Three ways to build a term:

Form	**Name**	**Reading**
x	variable	a placeholder
$(\lambda x.M)$	abstraction	"the function taking x to M"
$(M\ N)$	application	"apply M to N"

That is the entire syntax. Everything else is convention:

- Application associates left: $M\ N\ P$ means $((M\ N)\ P)$
- Abstraction extends as far right as possible: $\lambda x.M\ N$ means $\lambda x.(M\ N)$
- We write $\lambda xy.M$ for $\lambda x.\lambda y.M$

Examples. $\lambda x.x$ (identity) $\lambda x.\lambda y.x$ (first projection) $\lambda f.\lambda x.f(f\ x)$

Bound and Free

In $\lambda x.M$, the λx **binds** every free occurrence of x in M.

Term	**Bound**	**Free**
$\lambda x.x$	x	—
$\lambda x.y$	—	y
$\lambda x.(x\,y)$	x	y
$(\lambda x.x)\,x$	the first x	the last x

A term with no free variables is **closed**. The denotations we write for words will almost always be closed terms.

This distinction matters more than it looks. Variable binding is where the subtleties live – and it is one of the things Theme 5 has to rebuild from scratch, since the wedge product binds in a quite different sense.

Alpha: Names Do Not Matter

α-conversion

$\lambda x.M \equiv_\alpha \lambda y.M[x := y]$ provided y is not free in M

$\lambda x.x$ and $\lambda y.y$ are the same function. The bound variable is a placeholder, not a name.

Why the side condition? Without it:

$$\lambda x.\lambda y.x \quad \not\equiv \quad \lambda y.\lambda y.y$$

Renaming x to y would **capture** it – the outer variable gets swallowed by the inner binder, and a projection function silently becomes something else.

Capture-avoidance is the first place the lambda calculus makes you be careful. It is not the last.

Beta: Computation

β-reduction

$$(\lambda x.M)\, N \;\rightsquigarrow\; M[x := N]$$

Replace by N every **free occurrence of** x **in** M — equivalently, every occurrence in $\lambda x.M$ that *this* λ binds. Rename bound variables as needed to avoid capture.

This is the whole computational content of the calculus – and, for us, it *is* semantic composition.

Worked example.

$$(\lambda x.x)\, a \;\rightsquigarrow\; a$$

$$(\lambda x.\lambda y.x)\, a\, b \;\rightsquigarrow\; (\lambda y.a)\, b \;\rightsquigarrow\; a$$

$$(\lambda f.\lambda x.f\,(f\,x))\; g\; c \;\rightsquigarrow\; (\lambda x.g\,(g\,x))\; c \;\rightsquigarrow\; g\,(g\,c)$$

Free or Bound? A Question Worth Asking

Look again at the occurrences of x we are replacing.

$$(\ \underbrace{\lambda x.\ \overbrace{x\,(f\,x)}^{M}}_{\text{here } x \text{ is } \textit{bound}}\ )\ N$$

in M alone, x is *free*

Both descriptions are correct. They differ in *which term* you are asking about.

Asking about	**Status of x**	**Why**
$\lambda x.M$	**bound**	the binder is present
M alone	**free**	the binder has been stripped off

"Free" and "bound" are never absolute. An occurrence is free *in* a term — change the term and the answer changes. Substitution operates on M, not on $\lambda x.M$.

Why the Definition Must Say "Free"

It is tempting to write "replace every *bound* occurrence", since that is how they look in $\lambda x.M$. That wording gives wrong answers.

Test it on $(\lambda x.\ \lambda x.\ x)\ a$, where $M = \lambda x.\, x$:

Wording	**Selects**	**Result**	
"every *bound* occurrence of x in M"	the x in $\lambda x.x$ *is* bound	$\lambda x.\, a$	×
"every *free* occurrence of x in M"	x is not free in $\lambda x.x$	$\lambda x.\, x$	✓

The flaw: "bound" does not say *bound by what*. In $\lambda x.\lambda x.\, x$ the occurrence is bound — by the inner binder, which is none of our business. The word over-selects.

The wording that satisfies everyone: replace every occurrence of x that *this* λ binds — equivalently, every free occurrence of x in M. The demonstrative is what rules out the inner binder.

Capture in Action

Consider $(\lambda x.\lambda y.x\, y)\; y$.

Naive substitution – replace x by y directly:

$$\lambda y.y\, y \qquad \times$$

The argument y was free; now it is bound by the inner λy. We have changed the meaning.

Correct: α-convert the inner binder first, then substitute:

$$(\lambda x.\lambda z.x\, z)\; y \;\rightsquigarrow\; \lambda z.y\, z \qquad \checkmark$$

Every implementation of a semantics – including the one you write in Problem Set 1 – has to get this right. Haskell's binding structure gives it to you for free, which is one of several reasons we use it.

Eta, and What Reduction Guarantees

η-conversion

$\lambda x.(M\,x) \;\equiv_\eta\; M$ provided x is not free in M

A function that does nothing but pass its argument along *is* that function. (Extensionality.)

Two facts we will lean on all semester:

- **Church–Rosser (confluence).** If a term reduces two different ways, the results can always be brought back together. The order in which you perform composition does not change the result.
- **Normal forms.** A term in normal form has no reducible applications left. In the *typed* calculus we will use, every term has one and reduction always terminates.

Remember confluence. In Theme 5 we ask whether the geometric framework has an analogue – and, as the open-problems slide said, that answer is not yet known.

Currying: Functions of Several Arguments

The calculus has only one-argument functions. That is not a limitation.

Instead of a function of two arguments, take one argument and *return a function*:

$$\lambda x.\lambda y.M$$

Why this matters for semantics. Consider *likes*, a two-place relation:

$$[\![\text{likes}]\!] \;=\; \lambda x.\lambda y.\ \text{like}(y, x)$$

Apply it to the object first, and you get a one-place predicate – *likes Bob* – which is exactly the meaning of a verb phrase:

$$[\![\text{likes}]\!](\text{b}) \;\rightsquigarrow\; \lambda y.\ \text{like}(y, \text{b})$$

Currying is why English syntax – verb combines with object, then with subject – lines up with function application. Partial application *is* phrase structure.

A First Semantic Derivation

Alice likes Bob.

Lexicon. $[\![\text{Alice}]\!] = \text{a}$, $[\![\text{Bob}]\!] = \text{b}$, $[\![\text{likes}]\!] = \lambda x.\lambda y.\ \text{like}(y, x)$

Composition. Verb phrase first:

$$[\![\text{likes Bob}]\!] = (\lambda x.\lambda y.\ \text{like}(y, x))\ \text{b} \ \rightsquigarrow\ \lambda y.\ \text{like}(y, \text{b})$$

Then the subject:

$$[\![\text{Alice likes Bob}]\!] = (\lambda y.\ \text{like}(y, \text{b}))\ \text{a} \ \rightsquigarrow\ \text{like}(\text{a}, \text{b})$$

Two β-reductions, and we have a formula whose truth we can check against a model.

And *Bob likes Alice* yields $\text{like}(\text{b}, \text{a})$. The problem we opened with – solved, exactly, in two steps.

Practice

Reduce each to normal form. Work with a neighbor; five minutes.

1. $(\lambda x.\, x\, x)\ (\lambda y.\, y)$
2. $(\lambda x.\lambda y.\, y)\ a\ b$
3. $(\lambda f.\, f\, (f\, a))\ (\lambda x.\, x)$
4. $(\lambda x.\lambda y.\, x\, y)\ (\lambda z.\, z)\ c$

Practice

Reduce each to normal form. Work with a neighbor; five minutes.

1. $(\lambda x.\, x\, x)\ (\lambda y.\, y)$
2. $(\lambda x.\lambda y.\, y)\ a\ b$
3. $(\lambda f.\, f\, (f\, a))\ (\lambda x.\, x)$
4. $(\lambda x.\lambda y.\, x\, y)\ (\lambda z.\, z)\ c$

Answers.

1. $(\lambda y.y)(\lambda y.y) \rightsquigarrow \lambda y.y$
2. $(\lambda y.y)\, b \rightsquigarrow b$
3. $(\lambda x.x)((\lambda x.x)\, a) \rightsquigarrow (\lambda x.x)\, a \rightsquigarrow a$
4. $(\lambda y.\, (\lambda z.z)\, y)\, c \rightsquigarrow (\lambda z.z)\, c \rightsquigarrow c$

In (1), try reducing the inner and outer redexes in different orders. Same answer – that is confluence.

Where This Is Going

We can now build meanings and combine them. But nothing so far stops us writing nonsense:

$$[\![\text{Alice}]\!]\ ([\![\text{Bob}]\!]) \qquad — \qquad \text{apply an } \textit{individual} \text{ to an } \textit{individual}?$$

The untyped calculus will happily let us. Natural language will not.

Where This Is Going

We can now build meanings and combine them. But nothing so far stops us writing nonsense:

$$[\![\text{Alice}]\!]\ ([\![\text{Bob}]\!]) \qquad \text{—} \qquad \text{apply an } \textit{individual} \text{ to an } \textit{individual}?$$

The untyped calculus will happily let us. Natural language will not.

Next session: semantic types. We assign every expression a type, and composition is licensed only when the types fit. Ill-formed combinations stop being *wrong* and start being *impossible*.

That single move – arity and combinability tracked in the type – is precisely what Theme 5 reconstructs geometrically, where the type becomes a **grade**. Nearly everything in Session 2 has a counterpart in Session 18.

Before You Leave

Handout: Linear Algebra Self-Assessment

On paper, by the door. **Not graded, and not a test of whether you belong here.**

- Twenty-one questions, six sections. Expect 30–40 minutes.
- Work **without notes, references, or AI assistance** – a wrong answer costs you nothing and tells you something useful; a looked-up answer tells you nothing at all.
- Circle your confidence per section *before* checking answers.
- Answer key and review guide posted to the course site tonight.
- Report your six section scores via the form by **Monday night**. Two minutes. It determines how we pace Sessions 12–16.

Section E – determinants, area, and orientation – is the one that matters most. You saw why in Theme 3.

For Tuesday: EU Chapter 2; Jurafsky & Martin Appendix F. **Problem Set 1** goes out Friday.

Questions?

Next: Session 2 — Semantic Types and Type-Driven Interpretation